



Complexity Analysis

Computational Complexity

- The same problem can frequently be solved with different algorithms that differ in **efficiency**.
- *The differences between the algorithms may be immaterial for processing a small number of data items, but these differences grow proportionally with the **amount of data**.*
- *In order to compare the efficiency of algorithms, a measure of degree of difficulty of an algorithm is called the **computational complexity**.*

Computational Complexity

Efficiency Criteria

- **Computational Complexity** indicates how much effort is needed to apply an algorithm or how costly it is.
- Two efficiency criteria: **Time and Space**
- The factor of **time is more important** than that of space, so efficiency considerations usually focus on the amount of time elapsed when processing data. However, the most inefficient algorithm run on a Cray Computer can execute much faster than most efficient algorithm run on a PC, so run-time is always system dependent.

Computation Complexity

- To compare 100 algorithms all of them would have **to run on the same machine**. Furthermore, the results of run-time tests depend on the **language** in which a give algorithm is written even if the tests are performed on the same machine.
- To evaluate the efficiency, real time units such as microseconds and nanoseconds should not be used. Rather, **logical units** that express the relationship between the **size n** of a file or an array and the amount of **time t** required to process the data should be used.

Computational Complexity

• If n and t are linearly related $t_1 = c n_1$ then:

- If $n_2 = 5 n_1$ then $t_2 = 5 t_1$
- If $t_1 = \log_2 n$ the doubling n increases t by only one unit of time.
 - Therefore, $t_2 = \log_2 (2n)$, then $t_2 = t_1 + 1$
- A function expressing the relationship between n and t is usually much more complex and calculating such a function is important only in regard to large bodies of data.
- **Note:** Any term which does not substantially change the magnitude of the function should be eliminated from the function.



Computational Complexity

- A function expressing the relationship between n and t is usually much more complex and calculating such a function is important only in regard to large bodies of data.
- **Note:** Any term which does not substantially change the magnitude of the function should be eliminated from the function.

Asymptotic Complexity

- The resulting function only gives an approximate measure of efficiency of the original function
- this **approximate measure of efficiency** is called **asymptotic complexity**
- For example: $f(n) = n^2 + 100n + \log_{10}n + 1000$ (2.1)

For small values of n , the last term, 1000, is the largest. When n equals 10, the second ($100n$) and last (1000) terms are on equal footing with the other terms making a small contribution to the function value. When n reaches the value of 100, the first and the second terms make the same contribution to the result. But when n becomes larger than 100, the contribution of the second term becomes less significant. Hence, for large values of n , due to the quadratic growth of the first term (n^2), the value of the function f depends mainly on the value of this first term, as Figure 2.1 demonstrates. Other terms can be disregarded in the long run.

FIGURE 2.1 The growth rate of all terms of function $f(n) = n^2 + 100n + \log_{10}n + 1000$.

n	f(n)	n ²		100n		log ₁₀ n		1000	
	Value	Value	%	Value	%	Value	%	Value	%
1	1,101	1	0.1	100	9.1	0	0.0	1000	90.82
10	2,101	100	4.76	1,000	47.6	1	0.05	1000	47.62
100	21002	10,000	47.6	10,000	47.6	2	0.991	1000	4.76
1,000	1,101,003	1,000,000	90.8	100,000	9.1	3	0.0003	1000	0.09
10,000	101,001,004	100,000,000	99.0	1,000,000	0.99	4	0.0	1000	0.001
100,000	10,010,001,005	10,000,000,000	99.9	10,000,000	0.099	5	0.0	1000	0.00

Running time analysis

- The running time of an algorithm depends on:
 - how long it takes a computer to run the lines of code of the algorithm.
 - That in turn depends on the speed of the computer, the programming language, and the compiler that translates the program from the programming language into code that runs directly on the computer
 - as well as other factors.

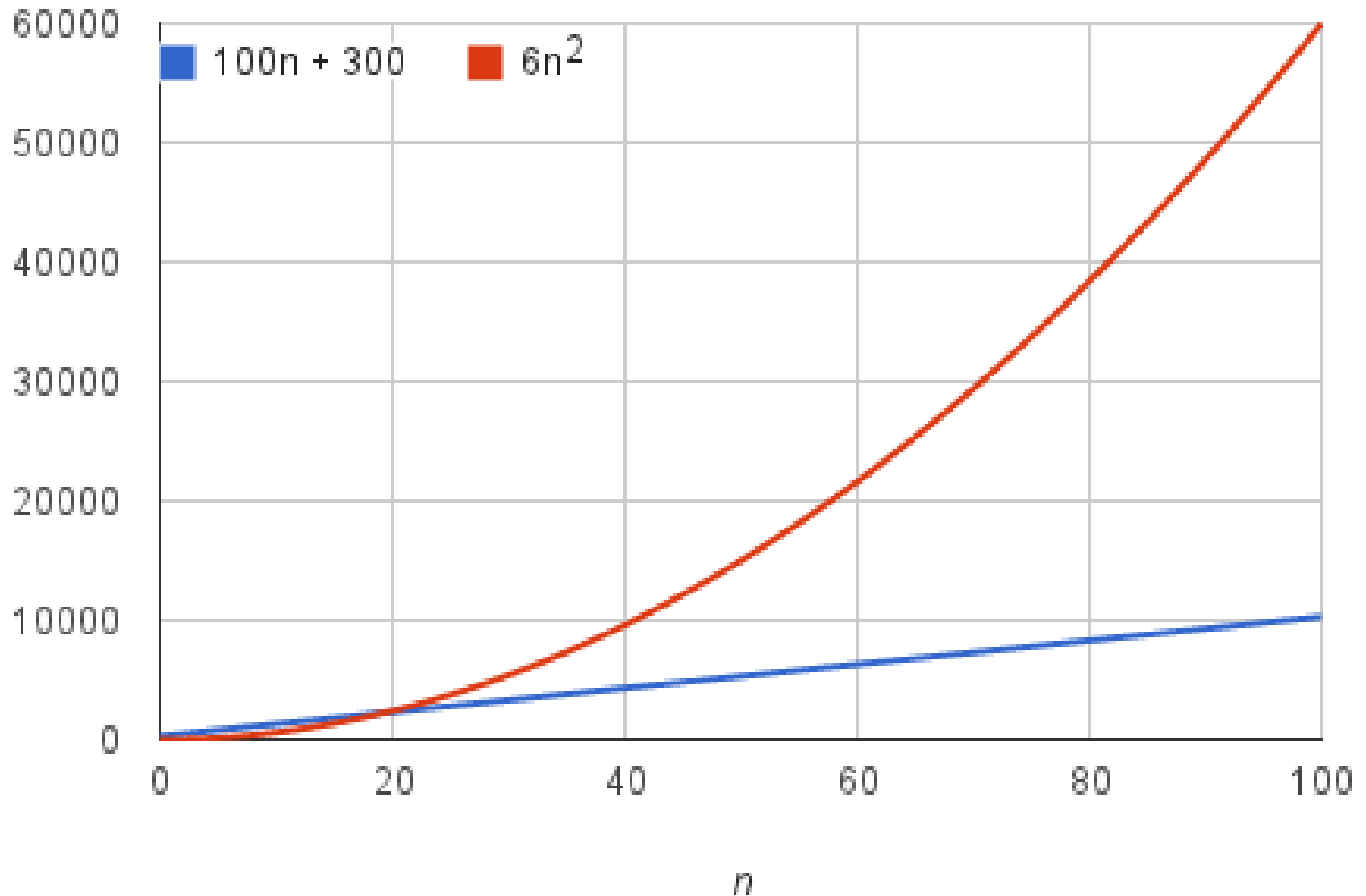
Running time analysis (2)

- Here we are more interested in running time in terms of **input size**
- we think about the running time of the algorithm *as a function of the size of its input*.
- Secondly we focus on how fast this function grows with the input size. We call that the **rate of growth** of the running time.
- So we simply propose the function to distill the most important part and cast aside the less important parts

Running time analysis (3)

- For example, suppose that an algorithm, running on an input of size n , takes **$6n^2 + 100n + 300$** machine instructions.
- The **$6n^2$** term becomes larger than the remaining terms, **$100n + 300$** once n becomes large enough (20 in this case).

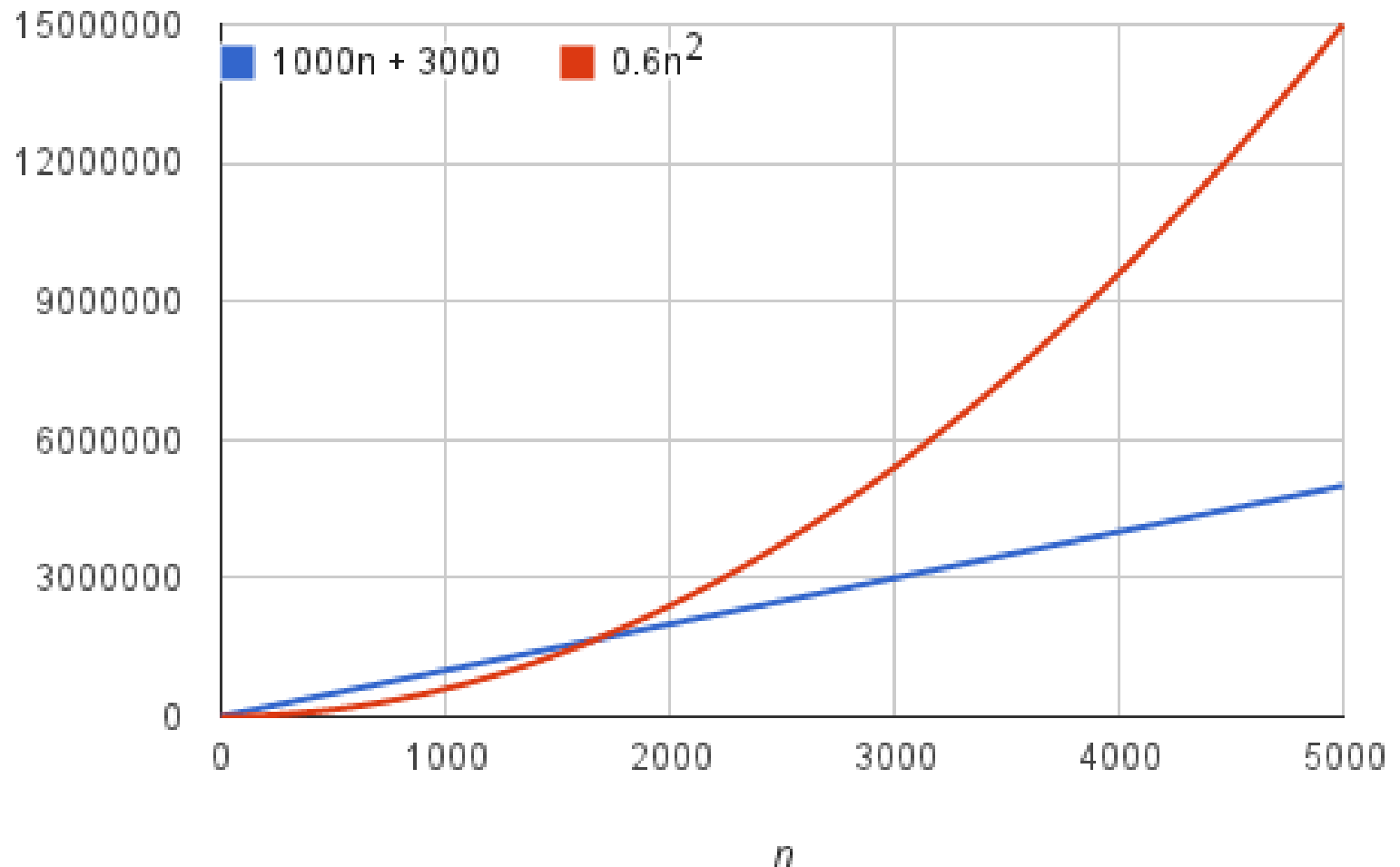
Running time analysis (4)



Running time analysis (5)

- We would say that the running time of this algorithm grows as n^2
- Why we ignored the coefficient 6 and the remaining terms $100n + 300$?
- As long as the running time is $an^2 + bn + c$ for some numbers $a > 0$, b , and c , there will always be a value of n for which an^2 is greater than $bn + c$
- this difference increases as n increases.
- **Example:** For values of $0.6n^2$ and $1000n + 3000$, we have:

Running time analysis (6)



Running time analysis (7)

- By dropping the less significant terms and the constant coefficients, we can focus on the important part of an algorithm's running time
 - its rate of growth
- When we drop the constant coefficients and the less significant terms, we use **asymptotic notation**.
- Three forms of it:
 - big- Θ notation
 - big- O notation , and
 - big Ω notation

Upper / lower bounds

- Big-O is the upper bound, while Omega is the lower bound.
- Theta requires both Big O and Omega
- For example, an algorithm taking $\Omega(n \log n)$ takes at least $n \log n$ time, but has no upper limit.
- An algorithm taking $\Theta(n \log n)$ is far preferential since it takes at least $n \log n$ ($\Omega(n \log n)$) and no more than $n \log n$ ($O(n \log n)$).

Big Theta (θ)

- Consider the **linear search algorithm**
- The maximum number of times that the for-loop can run is n , and this worst case occurs when the value being searched for is not present in the array
- Each step in the for loop takes a constant amount of time each time it executes.
- If the for-loop iterates n times, then the time for all n iterations is **$c1 \cdot n$**
 - where **$c1$** is the sum of the times for the computations in one loop iteration.

Big Theta (θ)

- Consider the **linear search algorithm**

```
var doLinearSearch = function(array) {  
  for (var guess = 0; guess < array.length; guess++) {  
    if (array[guess] === targetValue) {  
      return guess;  // found it!  
    }  
  }  
  return -1;  // didn't find it  
};
```

Big Theta (θ)

- The value of **$c1$** depends on the speed of the computer, the programming language used, the compiler or interpreter that translates the source program into runnable code,
 - and other factors
- If we also consider the other overhead time as **$c2$** , then total time for linear search in the **worst case** is **$c1.n + c2$**

Big Theta (θ)

- The constant factor **c1** and the low-order term **c2** don't tell us about **the rate of growth of the running time**.
- What's significant is that the worst-case running time of linear search grows like the array size ***n***.

Big Theta (θ)

- Consider the **linear search algorithm**

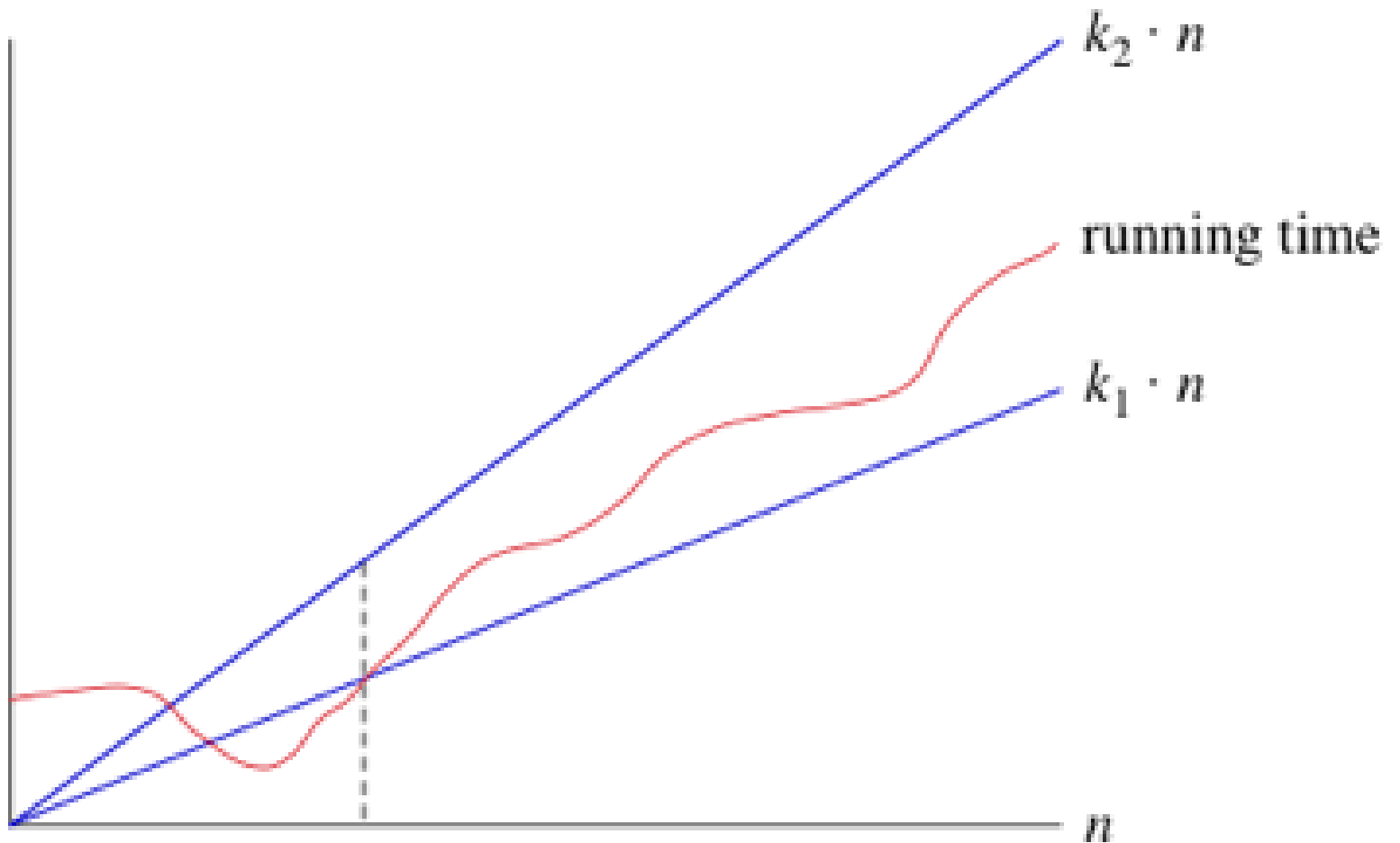
```
var doLinearSearch = function(array) {  
  for (var guess = 0; guess < array.length; guess++) {  
    if (array[guess] === targetValue) {  
      return guess; // found it!  
    }  
  }  
  return -1; // didn't find it  
};
```

This code has a little bit of extra overhead, for setting up the for-loop (including initializing guess to 0) and possibly returning -1 at the end. Let's call the time for this overhead

C_2 , which is also a constant. Therefore, the total time for linear search in the worst case is

$c_1 \cdot n + c_2$

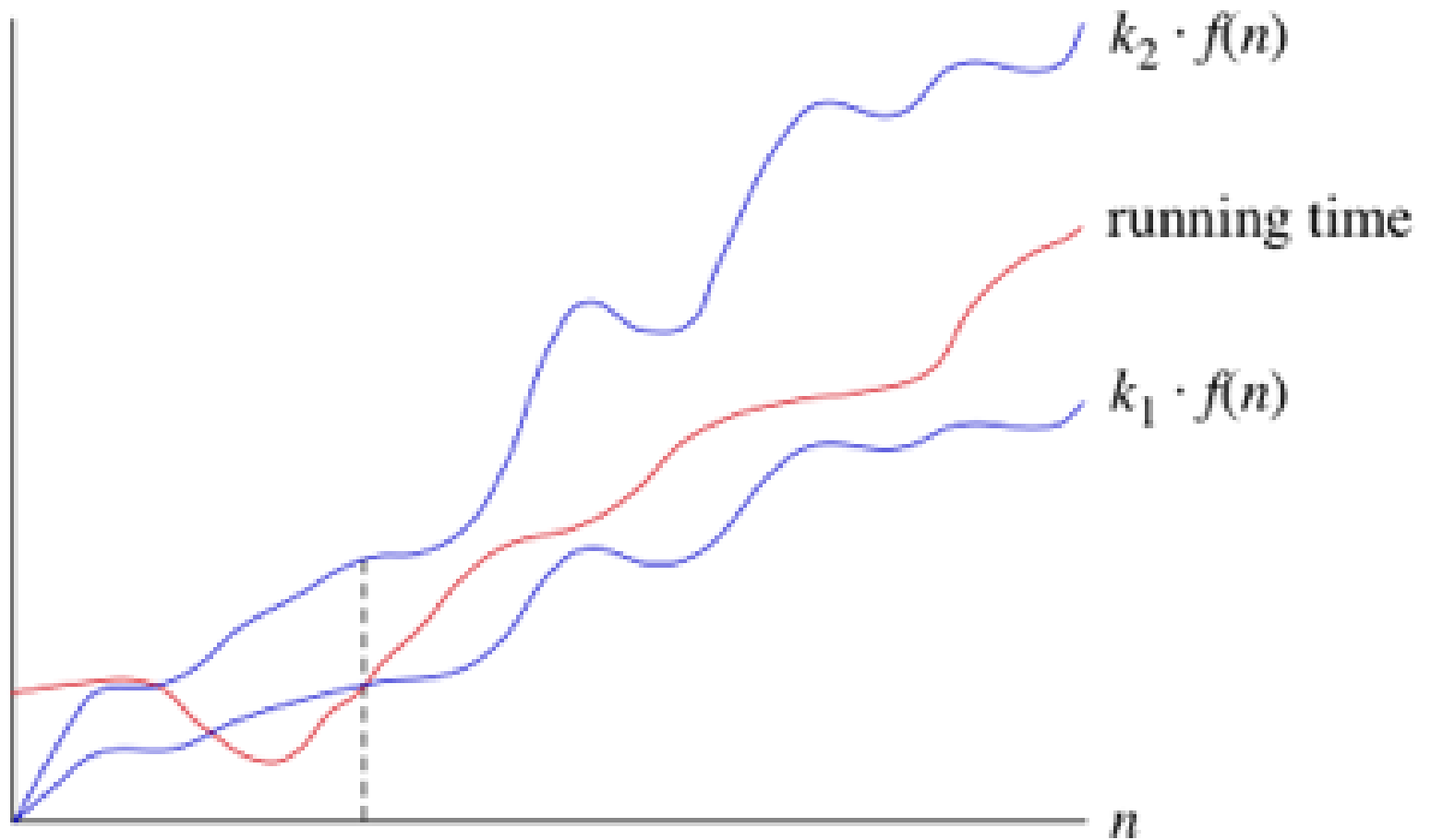
Big Theta (θ)



Big Theta (Θ)

- The notation $\Theta(n)$ called "big-Theta of n " or just "Theta of n ."
- When we say that a particular running time is $\Theta(n)$, we're saying that once n gets large enough, the running time is at least $k_1.n$ and at most $k_2.n$ for some constants k_1 and k_2
- For small values of n , we don't care how the running time compares with $k_1.n$ or $k_2.n$
- But once n gets large enough—on or to the right of the dashed line, the running time must be sandwiched between $k_1.n$ and $k_2.n$
- As long as these constants k_1 , k_2 exist, we say that the running time is $\Theta(n)$

For any function $f(n)$



Big Theta (θ)

- Once n gets large enough, the running time is between $k_1.f(n)$ and $k_2.f(n)$
- we just drop constant factors and low-order terms.
- Advantage of using big- Θ notation is that we don't have to worry about which time units we're using.

Big Theta (θ)

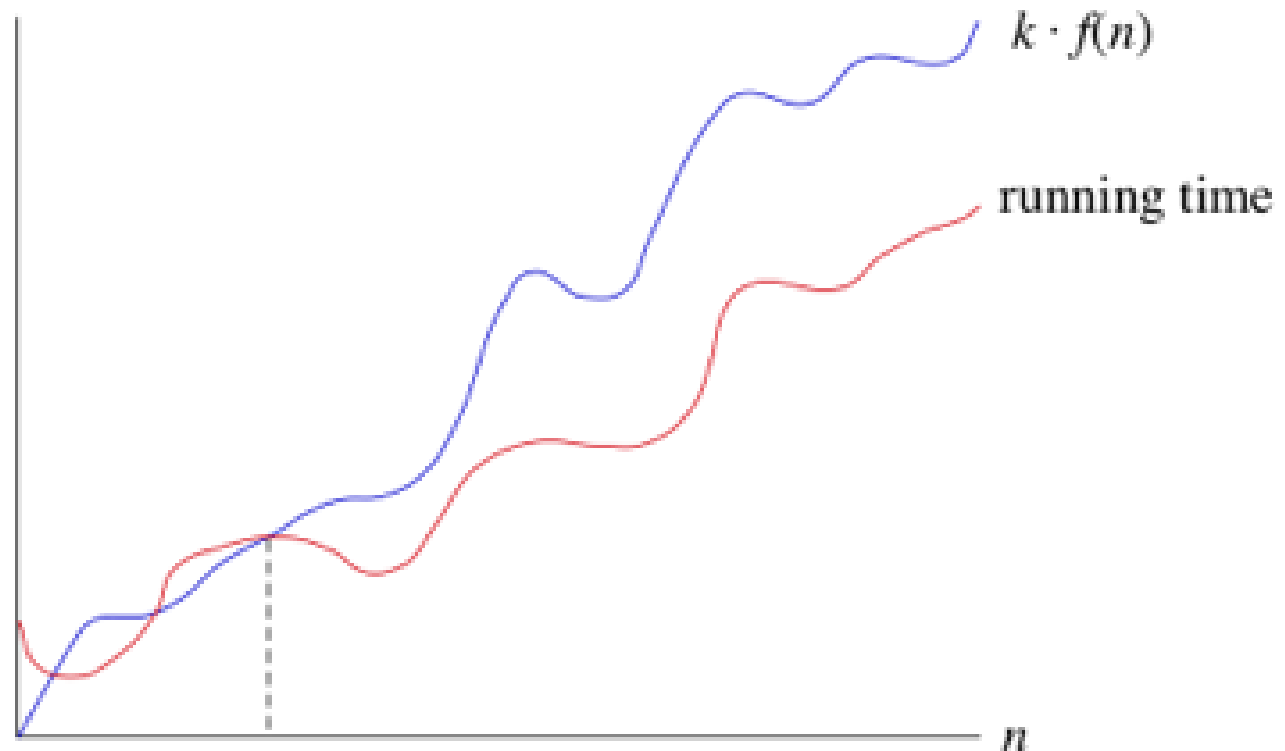
- For example, suppose that you calculate that a running time is $6n^2 + 100n + 300$ microseconds. Or maybe it's milliseconds.
- When you use big- Θ notation, you drop the factor 6 and the low-order terms $100n + 300$, and you just say that the running time is $\Theta(n^2)$.
- When we use big- Θ notation, we're saying that we have an asymptotically tight bound on the running time.
- "**Asymptotically**" because it matters for only large values of n
- "**Tight bound**" because we've nailed the running time to within a constant factor above and below.

Big-O notation

- Sometimes, we only need an **upper bound** on the running time
- **For example**, although the worst-case running time of binary search is $\Theta(\lg n)$, it would be incorrect to say that binary search runs in $\Theta(\lg n)$ time in all cases
- **What if we find the target value upon the first guess?**
- Then it runs in $\Theta(1)$ time
- The running time of binary search is never worse than $\Theta(\lg n)$, but it's sometimes better
- It would be convenient to have a form of asymptotic notation that means "the running time grows at most this much, but it could grow more slowly."

Big-O notation

- If a running time is $O(f(n))$, then for large enough n , the running time is at most $k \cdot f(n)$ for some constant k



Big-O notation

- We use big-O notation for **asymptotic upper bounds**, since it bounds the growth of the running time from above for large enough input sizes
- We can say that the running time of binary search is always $O(\lg n)$.
- Although the worst-case running time is $\Theta(\lg n)$
- But the strongest statement we can make is that binary search runs in $O(\lg n)$ time

Big-O notation

- big- Θ looks a lot like big-O notation, except that big- Θ notation bounds the running time from both above and below, rather than just from above
- If we say that a running time is $\Theta(f(n))$ in a particular situation, then it's also $O(f(n))$
- For example, we can say that because the worst-case running time of binary search is $\Theta(\lg n)$, it's also $O(\lg n)$
- The converse is not necessarily true: that binary search always runs in $O(\lg n)$ time but not that it always runs in $\Theta(\lg n)$ time

Big-O notation

- It is correct to say that binary search runs in $O(n)$ time
- That's because the running time grows no faster than a constant times n
- In fact, it grows slower
- Think of it this way
 - Suppose you have 10 dollars in your pocket. You go up to your friend and say, "I have an amount of money in my pocket, and I guarantee that it's no more than one million dollars." Your statement is absolutely true, though not terribly precise

Big- Ω (Big-Omega) notation

- If a running time is $\Omega(f(n))$, then for large enough n , the running time is at least $k \cdot f(n)$ for some constant k
- We use big- Ω notation for **asymptotic lower bounds**, since it bounds the growth of the running time from below for large enough input sizes
- Just as $\Theta(f(n))$ automatically implies $O(f(n))$, it also automatically implies $\Omega(f(n))$
 - So we can say that the worst-case running time of binary search is $\Omega(\lg n)$
- For example, just as if you really do have a million dollars in your pocket, you can truthfully say "I have an amount of money in my pocket, and it's at least 10 dollars,"

Big-Ω (Big-Omega) graphically

